A Project Report on

Supervised Learning Models to Predict the Yield Curve

Forecasting the 1-month, 5-year and 30-year US Treasury yields one week ahead with XGBoost, Keras, PyTorch (fastai), PyTorch (TabNet) and PyTorch (Lightning Flash)

Submitted by

Ashwin Anand (SRN: PES1UG24AM057)

Abhishek Mundanat (SRN: PES1UG24AM012)

Course: MLBD

Department of AIML, PES University

October 2026

Contents

Abstract **3**

1. Introduction **4**

1.1 The yield curve 4
1.2 Problem statement 4
1.3 Objectives 4
1.4 What this project adds 4

2. Methodology **6**

2.1 Data sources 6
2.2 Data preparation 7
2.3 Features 7
2.4 Targets and the three experiments 7
2.5 Train, validation and test split 8
2.6 Models 8
2.7 Evaluation metrics 9
2.8 Comparison framework 10
2.9 Tools and environment 10

3. Exploratory Data Analysis **11**

3.1 The yield curve from 2010 to 2026 11
3.2 Shapes and slope of the curve 12
3.3 Principal components 12
3.4 Relationships between features 13
3.5 How much yields move in a week 14
3.6 Level or change? 14
3.7 Data validation 15

4. Results **16**

4.1 Experiment A: why predicting the level fails 16
4.2 Main results (experiment C) 16
4.3 All three experiments 18
4.4 Statistical significance 19
4.5 When the models make errors 20
4.6 Explaining the models 21
4.7 Simplicity, transparency and efficacy 22
4.8 Practical problems and how they were solved 23

5. Conclusion and Summary **25**

5.1 Summary of findings 25
5.2 Limitations 25
5.3 Future work 25

References **26**

Appendix A: Project structure and how to run **27**

Appendix B: Chosen hyperparameters **28**

Abstract

This project predicts three points of the US Treasury yield curve, the 1-month (short term), 5-year (medium term) and 30-year (long term) yields, five trading days ahead. Daily data from January 2010 to September 2026 (4,189 trading days) was collected from Yahoo Finance and FRED. The predictors are the past yield curve, the shares of federal debt held by the public, foreign investors and the Federal Reserve, the Baa corporate credit spread, engineered curve-shape and momentum features, and market indicators from Yahoo Finance. Five frameworks were compared: XGBoost, Keras (TensorFlow), PyTorch with fastai, PyTorch with TabNet and PyTorch with Lightning Flash. A "no change" naive forecast and linear regression were used as baselines.

The exploratory analysis showed that yield levels are non-stationary and that the test period (2023-2026) had yields higher than anything seen in the training period. Predicting the yield level, as in the original case study, made every model fail; XGBoost was the worst with an average error of 137 basis points, because trees cannot extrapolate. Predicting the weekly **change** and removing the raw yield levels from the inputs brought all models to within about 1.7 basis points of the naive forecast. However, no model beat the naive forecast in a statistically significant way (Diebold-Mariano test), which suggests that one-week-ahead yield moves are close to unpredictable from public data. On simplicity, transparency and efficacy together, XGBoost ranked first (4.7/5), followed by TabNet, Keras, fastai and Lightning Flash.

1. Introduction

1.1 The yield curve

A yield curve is a line that plots the yields (interest rates) of bonds with the same credit quality but different maturities. The most watched yield curve is that of US Treasury bonds, which are considered free of credit risk. It is used as a benchmark for almost all other debt in the economy, such as mortgages, corporate loans and bank lending rates.

Normally the curve slopes upward: investors want a higher yield for lending money for a longer time. Sometimes the curve becomes flat or even **inverted**, with short-term yields above long-term yields. An inverted curve has often come before a recession (Estrella & Mishkin, 1998), so the slope of the curve is followed closely by investors, central banks and economists. Fixed income markets are an important source of finance for governments, banks and companies, and the yield curve is especially important for pension funds and insurance companies, whose liabilities are long term.

1.2 Problem statement

In a supervised regression setting, the task is to predict three tenors of the Treasury curve **five trading days (one week) ahead**:

- **1-month yield (DGS1MO):** short term, mainly driven by the Federal Reserve's policy rate
- **5-year yield (DGS5):** medium term
- **30-year yield (DGS30):** long term, driven by inflation and growth expectations and by the supply of long bonds

The predictors, as in the case study by Tatsat et al. (2020), are the previous values of the Treasury curve (1M, 3M, 1Y, 2Y, 5Y, 7Y, 10Y and 30Y), the percentage of federal debt held by the public, by foreign governments and by the Federal Reserve, and the corporate spread on Baa-rated debt relative to the 10-year Treasury yield.

1.3 Objectives

1. Connect to Yahoo Finance (and FRED) and build a clean daily dataset from 2010 onward.
2. Explore the data and understand what drives the yield curve.
3. Build regression models with XGBoost, Keras, PyTorch with fastai, PyTorch with TabNet and PyTorch with Lightning Flash.
4. Compare the frameworks on simplicity, transparency and efficacy, and on RMSE, MAE and R^2.
5. Report the results honestly, including where the models fail.

1.4 What this project adds

Besides implementing the case study with five frameworks, the project adds the following:

- **Two data sources with a cross-check.** Yahoo Finance does not have the 1-month yield or the debt data, so the curve comes from FRED, and Yahoo provides market-sentiment features. The yields available in both sources were compared.
- **No look-ahead bias in the quarterly debt data.** The values are only used from the date they would actually have been published.

- **Three experiments:** predicting the yield level (as in the case study), predicting the weekly change, and predicting the change without raw yield levels as inputs.
- **Honest benchmarks:** a naive "no change" forecast, the skill score against it, direction accuracy and a Diebold-Mariano significance test.
- **Explainability:** SHAP values for XGBoost, attention masks for TabNet and permutation importance for the other networks.
- **A measurable comparison rubric** for simplicity, transparency and efficacy.

2. Methodology

Table 1 shows the pipeline. Every notebook uses the same shared code (in the src folder) for loading the data, building features, splitting by date and computing metrics, so all models are compared under exactly the same conditions.

Table 1: Steps of the pipeline and where they are implemented

Step	What happens	Where
1. Collect	Download 13 FRED series and 9 Yahoo Finance tickers, save raw CSVs	src/data.py, notebook 01
2. Prepare	Align to one daily calendar, apply publication lag, forward fill, compute debt shares	src/data.py
3. Explore	Plots, PCA, stationarity tests, feature correlations	notebook 02
4. Features and split	23 features, 3 experiments, time-based train / validation / test split	src/features.py
5. Train	Naive, Linear, XGBoost, Keras, fastai, TabNet, Lightning Flash	notebooks 03-08
6. Evaluate	Same metrics for every model from saved prediction files	src/evaluate.py, notebook 09

2.1 Data sources

The assignment asks for the data to be extracted from Yahoo Finance. Yahoo publishes only four Treasury yield indices: ^IRX (13-week bill), ^FVX (5-year), ^TNX (10-year) and ^TYX (30-year). It does not have the 1-month yield, which is one of our three targets, or the 1-year, 2-year and 7-year yields, the federal debt data and the Baa spread. The original case study also took these series from FRED (Federal Reserve Economic Data). We therefore used both sources (Tables 2 and 3). FRED was downloaded from its public CSV link, which needs no API key; Yahoo Finance was accessed with the yfinance library. The data is frozen at 30 September 2026 so that all results can be reproduced.

Table 2: Series downloaded from FRED

Code	Description	Frequency
DGS1MO, DGS3MO, DGS1, DGS2, DGS5, DGS7, DGS10, DGS30	Treasury constant-maturity yields, 1 month to 30 years (%)	daily
BAA10Y	Moody's Baa corporate bond yield minus the 10-year Treasury yield (%)	daily
TREAST	Treasury securities held by the Federal Reserve ($ million)	weekly
FYGFDPUN	Federal debt held by the public ($ million)	quarterly
FDHBFIN	Federal debt held by foreign and international investors ($ billion)	quarterly
GFDEBTN	Total public debt ($ million)	quarterly

Table 3: Tickers downloaded from Yahoo Finance

Ticker	Description	Used for
^IRX, ^FVX, ^TNX, ^TYX	Treasury yield indices (13-week, 5Y, 10Y, 30Y)	cross-check with FRED
^VIX	CBOE volatility index ("fear index")	feature
^GSPC	S&P 500 index	feature (5-day return)

Ticker	Description	Used for
BZ=F	Brent crude oil futures	feature (5-day return)
GC=F	Gold futures	feature (5-day return)
DX-Y.NYB	US dollar index	feature (5-day return)

Brent was used instead of WTI crude oil because the WTI price became negative in April 2020, which makes percentage returns meaningless.

2.2 Data preparation

1. **Calendar.** The days on which Treasury yields were published (bond market trading days) form the master calendar. FRED marks holidays with blank values; these rows were removed.
2. **Publication delay.** FRED dates each quarterly debt value at the start of the quarter, but the number is published months later. For example, in October 2026 the latest total-debt value on FRED was still the one dated 1 April 2026. If the value were forward filled from its label date, the model would "know" debt numbers before they were released (look-ahead bias). The quarterly series were therefore moved forward by two quarters (three quarters for foreign holdings, which are released even later). The weekly Fed holdings for each Wednesday are released on Thursday afternoon, so they were moved forward by two days.
3. **Forward fill.** Weekly, quarterly and Yahoo values were carried forward until a new value was available.
4. **Debt shares.** As in the case study: GOV_PCT = TREAST / GFDEBTN, HOM_PCT = FYGFDPUN / GFDEBTN and FOR_PCT = FDHBFIN × 1000 / GFDEBTN (FDHBFIN is in billions, the others in millions), all in percent.

The final dataset has 4,189 daily rows from 2010-01-04 to 2026-09-30 and no missing values.

2.3 Features

Table 4 lists the 23 features. The first group is the 12 predictors from the case study; the other two groups were added in this project.

Table 4: Features used by the models

Group	Features	Count
Case study	8 yields (1M, 3M, 1Y, 2Y, 5Y, 7Y, 10Y, 30Y), GOV_PCT, HOM_PCT, FOR_PCT, BAA10Y	12
Curve shape and momentum	10Y-2Y slope, 10Y-3M slope, curvature (2×5Y - 2Y - 10Y), 5-day change of the 1M, 5Y and 30Y yields	6
Market (Yahoo Finance)	VIX level; 5-day % return of S&P 500, Brent, gold and the dollar index	5

2.4 Targets and the three experiments

The original case study predicts the yield level five days ahead. The EDA (Section 3.6) showed that yield levels are non-stationary and that the test period had yields far above the training period, which is a problem for any model that cannot extrapolate. We therefore ran every model in three

experiments (Table 5). In experiments B and C the predicted change is added to today's yield, so all experiments are evaluated on the same yield level.

Table 5: The three experiments

Experiment	Target	Inputs
A (case study)	yield level 5 trading days ahead	all 23 features
B	5-day change of the yield	all 23 features
C	5-day change of the yield	15 features: all except the 8 raw yield levels

Experiment C keeps only inputs that do not trend over time (spreads, recent changes, debt ratios and market indicators). The choice was checked on the validation year with linear regression: only the no-levels feature set gave positive validation skill for all three targets, so the test set was not needed to make this choice.

2.5 Train, validation and test split

Time series must never be split randomly, otherwise the model is trained on the future and tested on the past. The data was split by date: training 2010-01-11 to 2021-12-23 (2,993 rows), validation in 2022 (244 rows) and test from 2023-01-03 to 2026-09-23 (932 rows). The last five rows of the training and validation sets were dropped so that no five-day target window overlaps with the next set. The validation set was used only for early stopping and for choosing hyperparameters; the test set was used once, at the end. Scalers were fitted on the training data only.

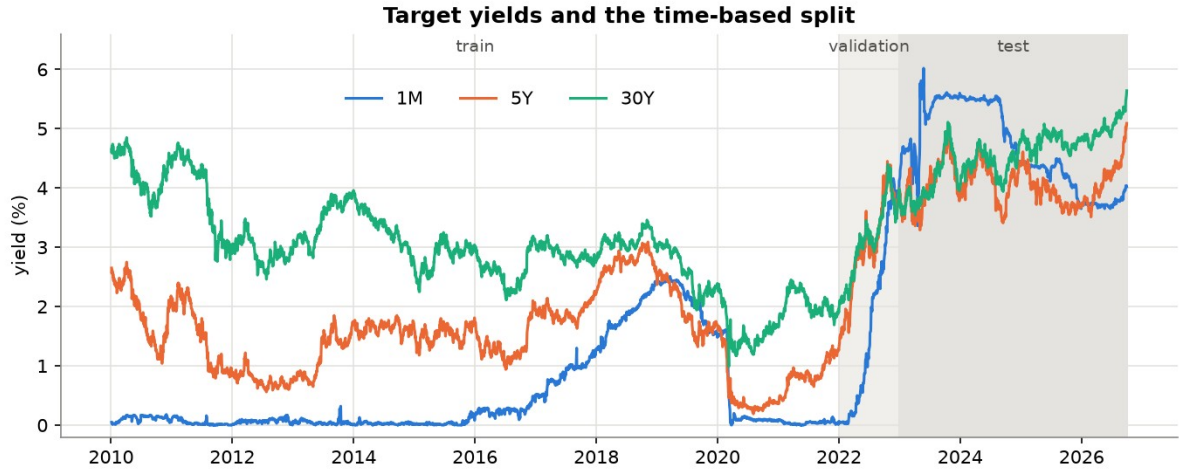


Figure 1: *The three target yields and the time-based split*

2.6 Models

Naive forecast (baseline)

The forecast for next week is simply today's yield. It has no parameters, needs no training and is the benchmark every model must beat.

Linear regression (baseline)

Ordinary least squares on standardised features (scikit-learn), one set of coefficients per target. It is the simplest real model and a natural benchmark for the five frameworks.

XGBoost

XGBoost (Chen & Guestrin, 2016) is a gradient boosting library: it builds many small decision trees one after another, where each new tree tries to correct the errors of the trees before it, and the prediction is the sum of all trees. One XGBRegressor was trained per target with early stopping (stop when the validation error has not improved for 100 rounds). A grid of 12 combinations of max_depth (2, 3, 5), learning_rate (0.01, 0.05) and min_child_weight (1, 20) was searched on the validation set, with subsample and colsample_bytree fixed at 0.8.

Keras (TensorFlow)

A multi-layer perceptron (MLP) with Dense layers, ReLU activations and one output neuron per target, so one network predicts all three targets. It was trained with the Adam optimiser and mean squared error loss, with early stopping (patience 20) restoring the best weights. Inputs and targets were standardised with training statistics. Six configurations were compared: hidden layers (32), (64, 32) or (128, 64), each with and without 20% dropout.

PyTorch with fastai

fastai (Howard & Gugger, 2020) is a high-level library on top of PyTorch. TabularPandas applies the Normalize step to the inputs automatically, tabular_learner builds an MLP with batch normalisation, lr_find suggests a learning rate, and fit_one_cycle trains with the one-cycle learning-rate policy (Smith, 2018), in which the learning rate first rises and then falls. Eight configurations were compared: layers [200, 100] or [64, 32], learning rate 10^{-3} or 10^{-2}, and 10 or 30 epochs.

PyTorch with TabNet

TabNet (Arik & Pfister, 2021) is a deep learning architecture designed for tabular data. It works in several decision steps; in each step an attentive transformer selects a small subset of features with a sparse mask (sparsemax), and a feature transformer processes only those features. The masks show which features the model used for every prediction, which makes TabNet more interpretable than a normal MLP. The pytorch-tabnet implementation was used with one model for all three targets, early stopping (patience 30) and four configurations: n_d = n_a = 8 or 16, and n_steps = 3 or 5.

PyTorch with Lightning Flash

Lightning Flash was a high-level task library from the PyTorch Lightning team. A TabularRegressor was built from a TabularRegressionData object with the category_embedding backbone (a plain MLP from the PyTorch Tabular library; Joseph, 2021), so that it is different from the TabNet model. Flash accepts only one target column, so one model was trained per target. Four configurations were compared: learning rate 10^{-3} or 10^{-2}, and 10 or 30 epochs. Flash was archived by Lightning AI in 2023 and needs old versions of its dependencies, so it was run in a separate Python 3.10 environment (Section 2.9).

All neural networks (Keras, fastai, TabNet and Flash) give slightly different results for different random seeds, so the final model of each was trained with three seeds (42, 7 and 2024) and the average is reported. Each framework had a similar, small tuning budget (4 to 12 configurations), chosen on the validation set with seed 42. The chosen settings are listed in Appendix B.

2.7 Evaluation metrics

All metrics are computed on the yield level in the test period, separately for 1M, 5Y and 30Y. RMSE and MAE are reported in basis points (1 bp = 0.01 percentage point).

- **RMSE** (root mean squared error) and **MAE** (mean absolute error): the size of the forecast errors. RMSE penalises large errors more.
- **R^2** on the yield level, as required. Because yields barely move in a week, even the naive forecast scores a high R^2 here, so this metric alone is misleading.
- **R^2 on the change:** how much of the actual weekly move the model explains. It is 0 for a constant forecast and negative if the model is worse than that.
- **Skill vs naive** = 1 - RMSE(model) / RMSE(naive). A positive value means the model beats the naive forecast.
- **Direction accuracy:** the percentage of days on which the model predicted the right direction (up or down) of the yield.
- **Diebold-Mariano test** (Diebold & Mariano, 1995): checks whether the squared errors of a model are significantly different from those of the naive forecast. The variance was corrected for the overlap of the five-day forecasts with the Newey-West method (Newey & West, 1987) using 4 lags.

2.8 Comparison framework

Simplicity, transparency and efficacy were each turned into a score from 1 (worst) to 5 (best) for the five frameworks, using rules that can be repeated:

- **Simplicity:** each framework was ranked on four measures: lines of code needed to build, train and predict (counted automatically from the training function in each notebook), number of hyperparameters we had to set, training time for one run, and setup effort (1 = installed with pip in the main environment, 3 = needed a separate Python version, old pinned versions and fixes). The average rank was mapped linearly to 1-5.
- **Transparency:** one point each for built-in global feature importance, built-in explanation of a single prediction, a model that can be inspected directly, and the same result on every run (half a point if partly true). Score = 1 + points.
- **Efficacy:** ranked on the average skill vs naive and the average direction accuracy in experiment C, with the average rank mapped to 1-5.

2.9 Tools and environment

Everything ran on the CPU of an Apple Silicon Mac with 8 GB of RAM. The main environment used Python 3.12 with pandas 3.1, scikit-learn 1.9, XGBoost 3.4, TensorFlow 2.21 with Keras 3.15, PyTorch 2.14, fastai 2.8, pytorch-tabnet 4.1, SHAP 0.54, statsmodels 0.15 and yfinance 1.7. Lightning Flash 0.8.2 requires pytorch-lightning below 2.0, so it cannot be installed next to the newer PyTorch used by fastai; it ran in a second environment with Python 3.10, PyTorch 1.13.1, pytorch-lightning 1.9.5 and pandas 1.5.3. Both environments are described by requirements files in the repository.

3. Exploratory Data Analysis

3.1 The yield curve from 2010 to 2026

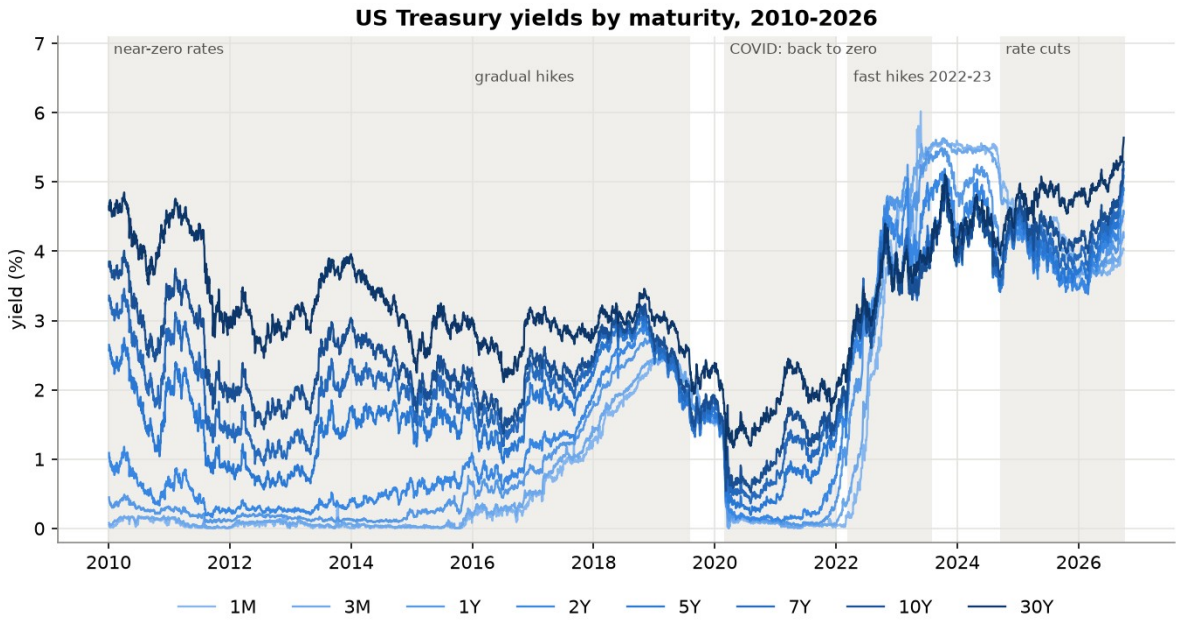


Figure 2: *US Treasury yields for all 8 maturities, with the main interest-rate periods shaded*

Figure 2 shows very different periods. From 2010 to 2015 the Federal Reserve kept its rate at zero after the 2008 crisis, so the 1M and 3M yields were close to 0% for six years while long-term yields still moved. Gradual rate hikes followed from 2016 to 2019, and by 2019 the curve was almost flat. COVID pushed all yields down in 2020; the 10Y reached its lowest point, 0.52%, in August 2020. In 2022-2023 the fastest hikes in decades took the 1M yield from about 0% to above 5% in roughly 18 months. In May 2023 it jumped to 6.02% during the debt-ceiling standoff. From late 2024 rate cuts brought short-term yields down to about 4%, while long-term yields kept rising; the 30Y reached 5.64% in September 2026, the highest value in the dataset.

3.2 Shapes and slope of the curve

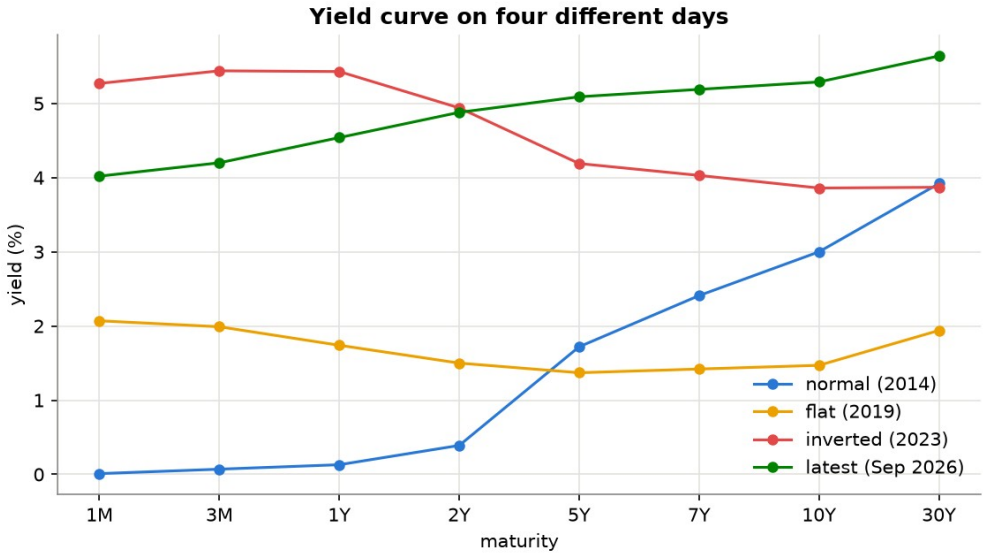


Figure 3: *The yield curve on four days: normal (2014), flat (2019), inverted (2023) and the latest (2026)*

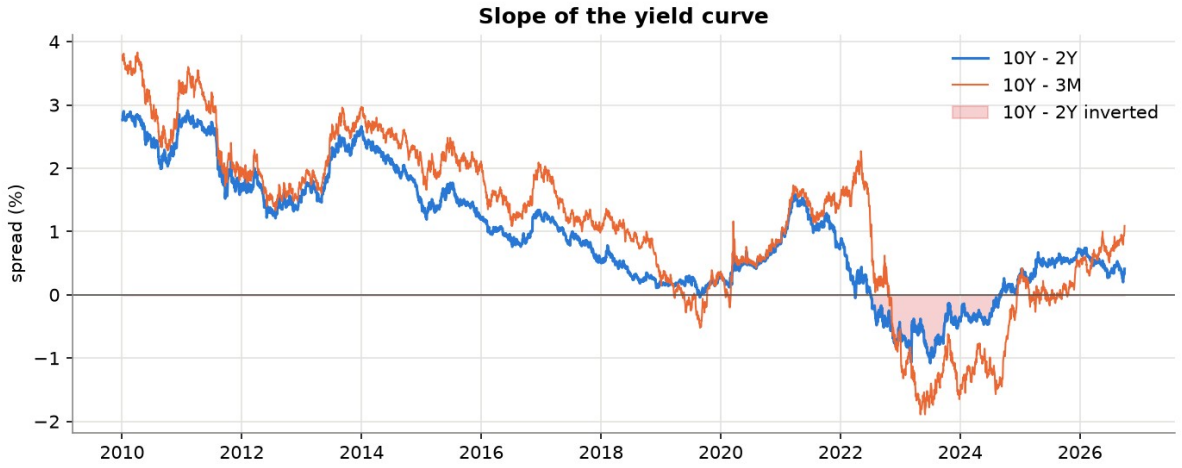


Figure 4: *Slope of the yield curve; the shaded area marks the inversion of the 10Y-2Y spread*

The 10Y-2Y spread was negative (inverted) from 6 July 2022 to 26 August 2024, 537 trading days and as deep as -1.08%. This was by far the longest inversion in the data. By 2026 both spreads were positive again because short rates fell with the rate cuts while long rates rose. The slope changes slowly over months, so it describes the current regime more than next week's move.

3.3 Principal components

PCA on the daily changes of the 8 yields (Figure 5) shows that three components explain 93.2% of all movement: level (73.6%, all maturities from 1Y to 30Y move together), slope (10.4%) and curvature (9.1%). This matches the classic result of Litterman & Scheinkman (1991). Our share is a little lower than the usual 99% because the 1M bill has a lot of its own movement (for example during the debt-ceiling episodes), which shows up as large 1M loadings in PC2 and PC3. For modelling, it means that the 8 yield features carry largely the same information.

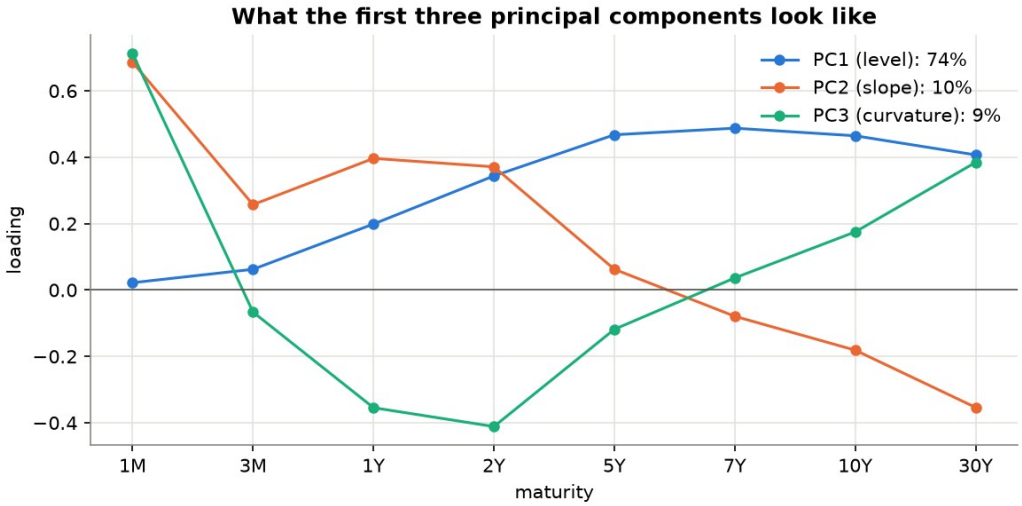


Figure 5: *Loadings of the first three principal components of daily yield changes*

3.4 Relationships between features

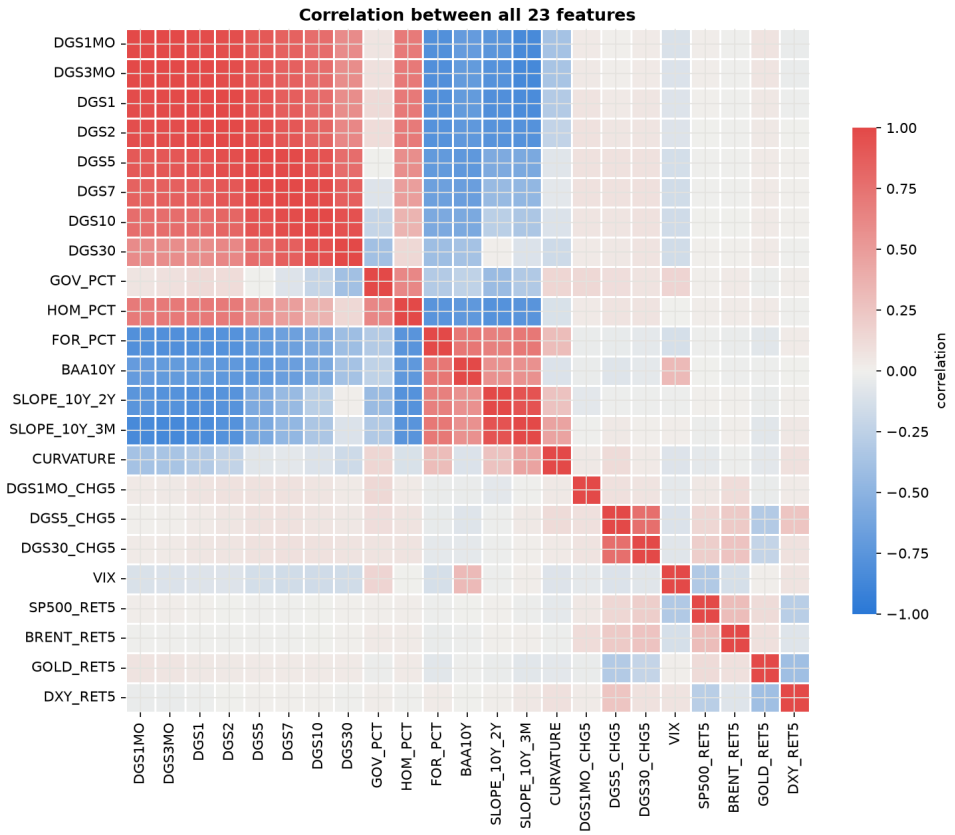


Figure 6: *Correlation between all 23 features*

The 8 yields are strongly correlated with each other (red block in Figure 6). The debt shares are also correlated with the yield levels because both trend over time; the foreign share fell as rates rose. The engineered changes and the market returns are hardly correlated with the levels, so they add new information. The high correlation between inputs is not a problem for trees and neural networks, but it makes linear regression coefficients unstable.

The correlation of each feature with the next week's change (training data only) was weak for every feature. The largest was 0.28, for the 1M yield's own change over the previous week (momentum). For the 5Y and 30Y nothing exceeded 0.13 in absolute value. This was a first sign that week-ahead changes would be hard to predict.

3.5 How much yields move in a week

Table 6: Statistics of the 5-day change of each target (2010-2026)

Target	Std. dev. (bp)	Mean abs. change (bp)	Weeks with no change	Kurtosis
1M	9.1	3.9	19%	56.9
5Y	11.5	8.5	5%	2.2
30Y	11.0	8.4	4%	1.4

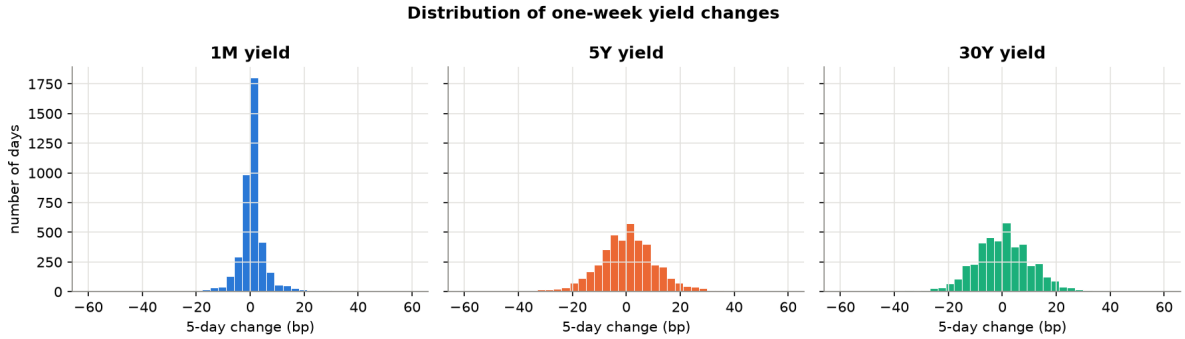


Figure 7: *Distribution of one-week yield changes*

A typical one-week move is about 9-12 basis points. The 1M yield is unusual: it did not move at all in 19% of weeks (when it was stuck at zero), but it also has very large jumps (kurtosis 57, against 0 for a normal distribution), because short-term rates move in steps when the Fed changes its policy rate.

3.6 Level or change?

Table 7: Stationarity (ADF test p-value) and autocorrelation at lag 5

Target	ADF p-value (level)	ADF p-value (5-day change)	Autocorr. lag 5 (level)	Autocorr. lag 5 (change)
1M	0.883	0.000	0.999	0.262
5Y	0.930	0.000	0.996	-0.014
30Y	0.762	0.000	0.993	-0.079

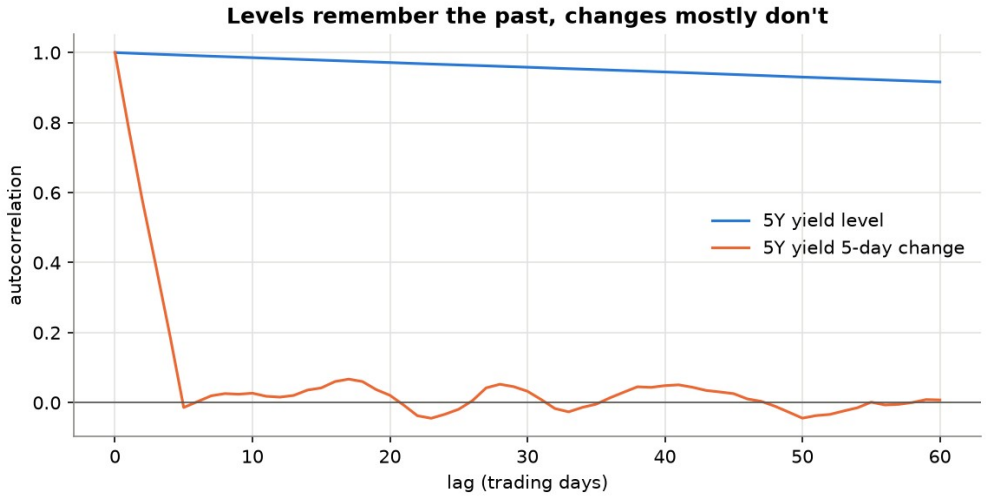


Figure 8: *Autocorrelation of the 5Y yield level and of its 5-day change*

Table 8: Range of each target in the training and test periods (%)

Target	Train min	Train max	Test min	Test max	Test days above train max
1M	0.00	2.51	3.36	6.02	100%
5Y	0.19	3.09	3.29	5.09	100%
30Y	0.99	4.85	3.54	5.64	26%

This is the key result of the EDA. Yield **levels** are not stationary (ADF p-values between 0.76 and 0.93) and their autocorrelation at lag 5 is above 0.99, so the value next week is almost the value today. A model that copies today's yield will already reach an R^2 close to 1, so a high R^2 on levels says little. Weekly **changes** are stationary ($p \approx 0$) and almost uncorrelated over time.

Moreover, every test day for the 1M and 5Y yields, and 26% of test days for the 30Y, are above the highest value in the training data (Table 8). Tree models such as XGBoost predict by averaging training targets in their leaves, so they cannot output values above the training maximum. This led to the three experiments described in Section 2.4.

3.7 Data validation

Table 9: Comparison of Yahoo Finance and FRED for the tenors in both sources

Yahoo	FRED	Correlation	Mean abs. diff. (bp)	Max abs. diff. (bp)
Yahoo 3M	DGS3MO	0.9999	6.37	40.7
Yahoo 5Y	DGS5	1.0000	0.87	13.3
Yahoo 10Y	DGS10	0.9999	0.99	10.5
Yahoo 30Y	DGS30	1.0000	0.65	12.7

For the 5Y, 10Y and 30Y yields the two sources match almost perfectly: correlation of at least 0.9999 and an average difference below one basis point. The small spikes happen on volatile days, because Yahoo's index closes at a slightly different time from the Treasury's official end-of-day curve. ^IRX differs more (about 6 bp) because it is a different instrument, a 13-week bill quoted as a discount rate. The data is therefore reliable.

4. Results

4.1 Experiment A: why predicting the level fails

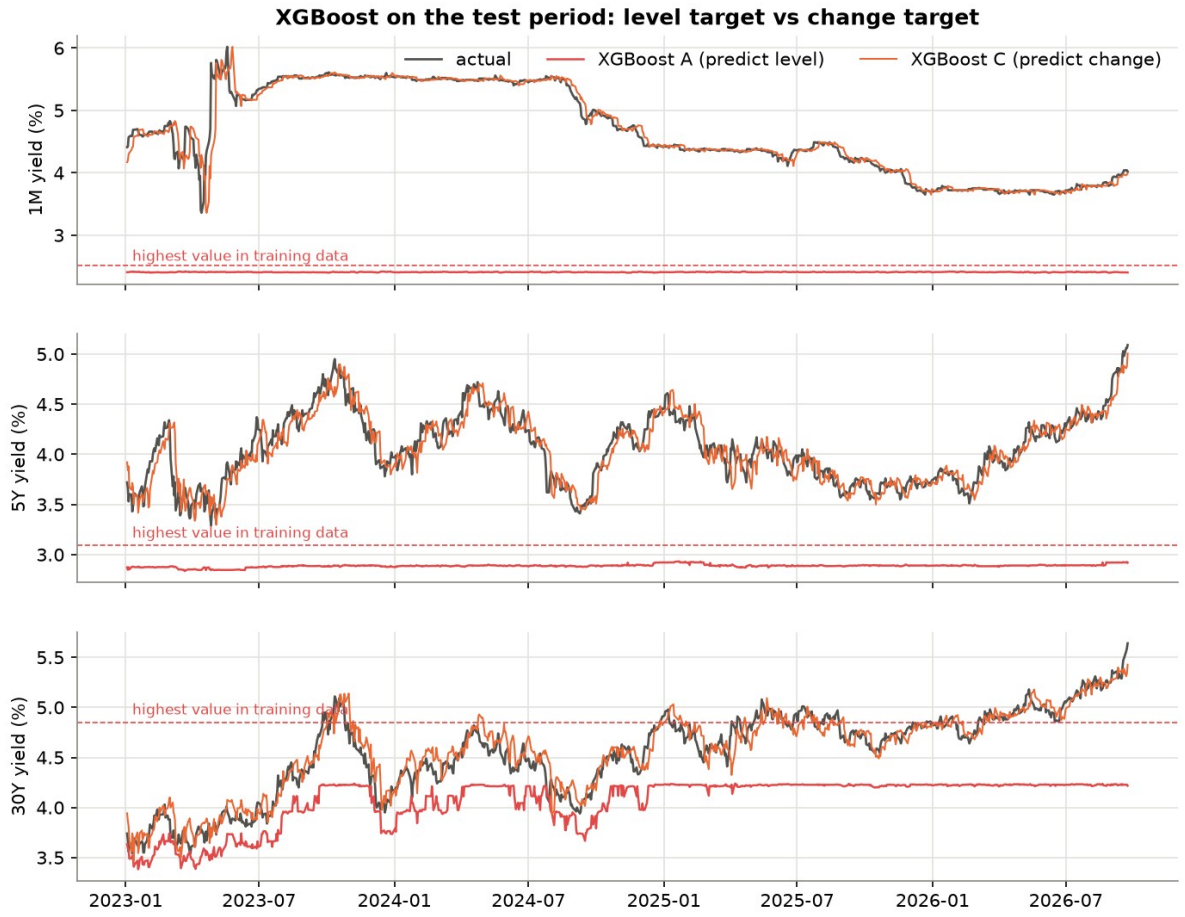


Figure 9: *XGBoost on the test period: experiment A (level target) against experiment C (change target). The dashed line is the highest value in the training data.*

Figure 9 confirms the EDA. When XGBoost predicts the level, its forecasts cannot go above the training range: they stay around 2.4% for the 1M while the real yield was between 3.4% and 6%, around 2.9% for the 5Y and are capped at about 4.2% for the 30Y. The test RMSE was 236 bp (1M), 122 bp (5Y) and 54 bp (30Y), with skill between -3.8 and -15.1. The neural networks can output values outside the training range, but they did not learn the right relationship for yield levels they had never seen and also failed (average RMSE between 43 and 87 bp). In experiment C the forecasts follow the real yield because they are today's yield plus a small predicted change.

4.2 Main results (experiment C)

Table 10: Test results of experiment C (change target, no raw yield levels). Neural networks are averaged over 3 seeds.

Model	Target	RMSE (bp)	MAE (bp)	R^2	R^2 change	Skill vs naive	Direction acc.
Naive	1M	14.7	6.2	0.956	-0.000	0.000	-
Naive	5Y	13.8	10.5	0.840	-0.002	0.000	-
Naive	30Y	11.1	8.7	0.932	-0.008	0.000	-
Linear	1M	15.0	7.4	0.954	-0.043	-0.021	57.3%

Model	Target	RMSE (bp)	MAE (bp)	R^2	R^2 change	Skill vs naive	Direction acc.
Linear	5Y	14.0	10.6	0.836	-0.026	-0.012	51.4%
Linear	30Y	11.5	9.2	0.927	-0.084	-0.037	48.7%
XGBoost	1M	14.7	6.2	0.956	0.000	0.000	49.3%
XGBoost	5Y	14.0	10.8	0.835	-0.033	-0.015	51.3%
XGBoost	30Y	12.7	9.7	0.912	-0.313	-0.141	53.9%
Keras	1M	15.8	7.1	0.949	-0.158	-0.074	53.1%
Keras	5Y	14.4	11.1	0.825	-0.099	-0.046	48.6%
Keras	30Y	12.5	9.9	0.913	-0.296	-0.124	48.1%
fastai	1M	16.7	8.1	0.943	-0.299	-0.139	52.0%
fastai	5Y	14.5	11.0	0.824	-0.102	-0.048	49.2%
fastai	30Y	12.0	9.6	0.920	-0.185	-0.084	47.0%
TabNet	1M	18.1	7.6	0.933	-0.515	-0.230	53.7%
TabNet	5Y	14.3	10.9	0.829	-0.074	-0.035	51.2%
TabNet	30Y	12.3	9.3	0.917	-0.228	-0.103	49.3%
Flash	1M	17.5	7.4	0.937	-0.424	-0.192	52.0%
Flash	5Y	14.5	11.1	0.823	-0.111	-0.053	48.6%
Flash	30Y	12.2	9.6	0.918	-0.217	-0.099	47.7%

No model beat the naive forecast on average. The average RMSE over the three targets was 13.2 bp for the naive forecast, 13.5 for linear regression, 13.8 for XGBoost, 14.2 for Keras, 14.4 for fastai, 14.7 for Lightning Flash and 14.9 for TabNet. The only positive skill (XGBoost for the 1M, +0.0001) comes from a model that consists of a single tree and predicts almost no change, which is effectively the naive forecast.

R^2 on levels was between 0.82 and 0.96 for every model, including the naive forecast, which confirms that it is not informative here. R^2 on the change was negative for every model. Direction accuracy was between 47% and 57%; the best was linear regression for the 1M yield (57%). Overall the simpler models came closest to the naive forecast: linear regression and XGBoost (skill -0.02 and -0.05 on average) did better than all four neural networks (-0.08 to -0.12).

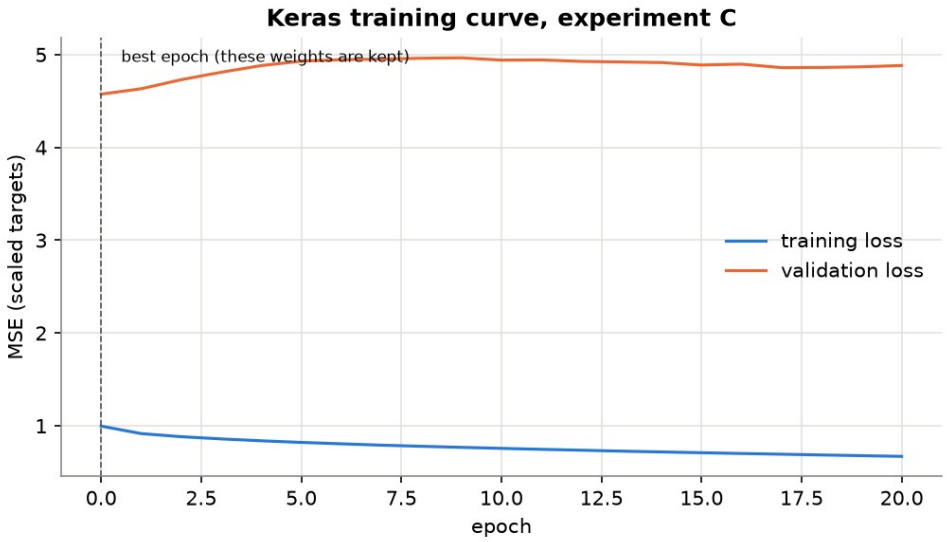


Figure 10: *Keras training curve (experiment C): the validation loss is lowest after the first epoch*

The training curves explain why the neural networks end up close to the naive forecast. For Keras (Figure 10) the validation loss was lowest right after the first epoch and then increased while the training loss kept falling: whatever the network learned about 2010-2021 made its 2022 predictions worse. Early stopping therefore kept an almost untrained network that predicts close to the average change. fastai and TabNet showed the same behaviour, and the fastai learning-rate finder found no learning rate at which the loss clearly decreased.

4.3 All three experiments

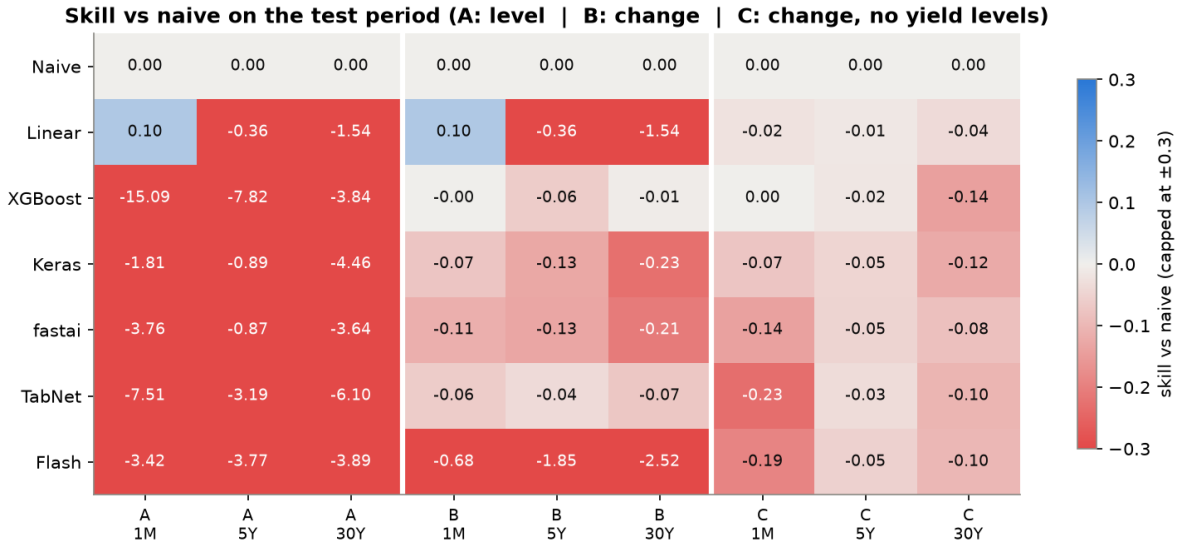


Figure 11: *Skill vs naive for every model, experiment and target (blue = better than naive, red = worse; colours capped at ±0.3)*

Table 11: Average test RMSE over 1M, 5Y and 30Y (bp)

Model	A: level, all features	B: change, all features	C: change, no yield levels
Naive	13.2	13.2	13.2
Linear	20.1	20.1	13.5
XGBoost	137.2	13.5	13.8

Model	A: level, all features	B: change, all features	C: change, no yield levels
Keras	42.6	15.0	14.2
fastai	49.1	15.1	14.4
TabNet	87.2	13.9	14.9
Flash	61.7	34.3	14.7

Experiment A was far worse than naive for every learned model. For linear regression A and B gave identical results; this is expected, because today's yields are inputs, so predicting the future yield or its change is the same least-squares problem. In experiment B, XGBoost and TabNet were close to naive but Lightning Flash failed (skill -0.7 to -2.5). Experiment C gave the most consistent results, with all models in a narrow band between -0.23 and 0. Going from A to C reduced the error of every learned model by 1.5 to 10 times.

4.4 Statistical significance

Table 12: Diebold-Mariano test against the naive forecast (experiment C, seed 42). Negative = smaller errors than naive.

Model	1M stat.	1M p	5Y stat.	5Y p	30Y stat.	30Y p
Linear	0.58	0.565	1.32	0.186	2.58	0.010
XGBoost	-0.32	0.748	1.38	0.166	3.70	0.000
Keras	0.74	0.462	2.43	0.015	1.06	0.289
fastai	2.27	0.023	2.36	0.018	3.88	0.000
TabNet	1.77	0.077	1.94	0.052	2.56	0.011
Flash	1.88	0.060	1.64	0.102	3.13	0.002

None of the 18 model-target pairs was significantly better than the naive forecast (0 of 18), while 8 of 18 were significantly worse at the 5% level, mostly for the 30Y yield (5 of 6 models). Statistically, the best result is that some models are as good as the naive forecast. This agrees with earlier research in which a random walk is very hard to beat at short horizons (Diebold & Li, 2006), and with the idea that bond markets quickly price in public information.

4.5 When the models make errors

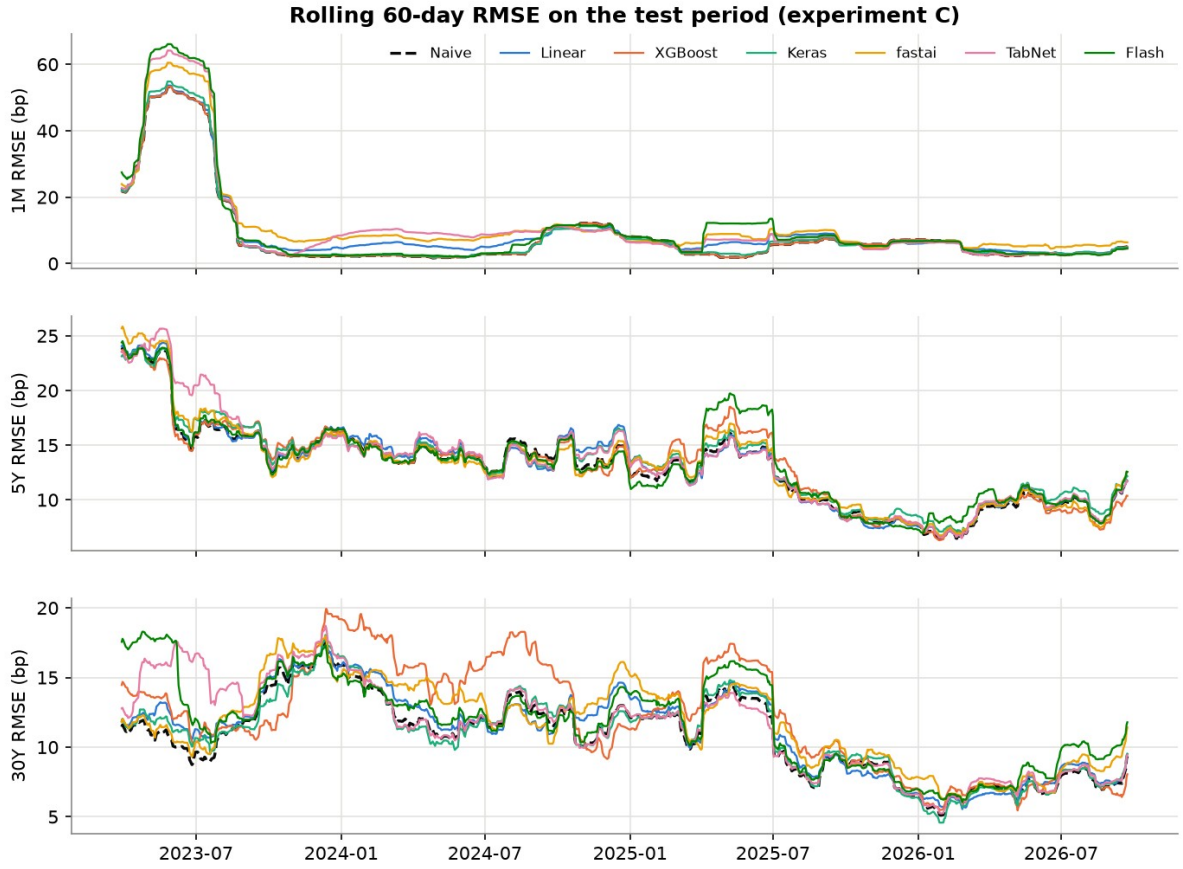


Figure 12: *Rolling 60-day RMSE of every model in the test period (experiment C)*

The errors were largest in spring 2023. The rolling 1M RMSE rose above 50-65 bp during the March 2023 banking stress after the failure of Silicon Valley Bank and the May 2023 debt-ceiling standoff. Every model, including the naive forecast, missed these shocks. The 1M RMSE fell from about 27 bp in 2023 to about 4 bp in 2026 as markets calmed down. The differences between models are small compared with how much the error changes over time: the market environment mattered more than the choice of model.

4.6 Explaining the models

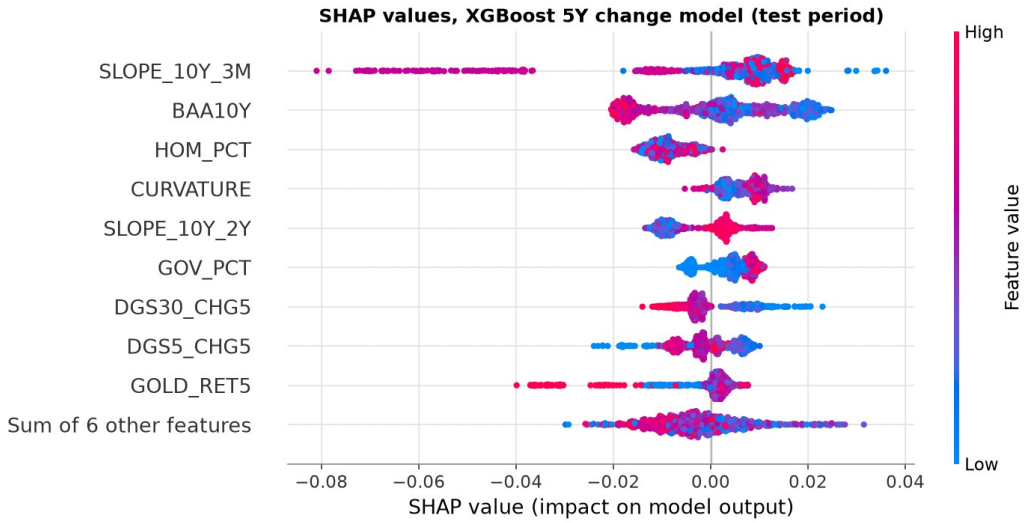


Figure 13: *SHAP values of the XGBoost 5Y change model on the test period*

SHAP values (Lundberg & Lee, 2017) split every prediction into a contribution from each feature. For the XGBoost 5Y model (Figure 13) the 10Y-3M slope and the Baa credit spread had the largest effects. A wide credit spread pushed the prediction down, which matches economic intuition: in stressful times investors buy government bonds and expect rate cuts, so yields fall. Most SHAP values were within ±2 bp, small compared with a typical weekly move of 11 bp. For one example day, 15 November 2024, the model predicted -12.3 bp for the 5Y, mainly because of the inverted 10Y-3M slope, while the actual move was 0 bp.

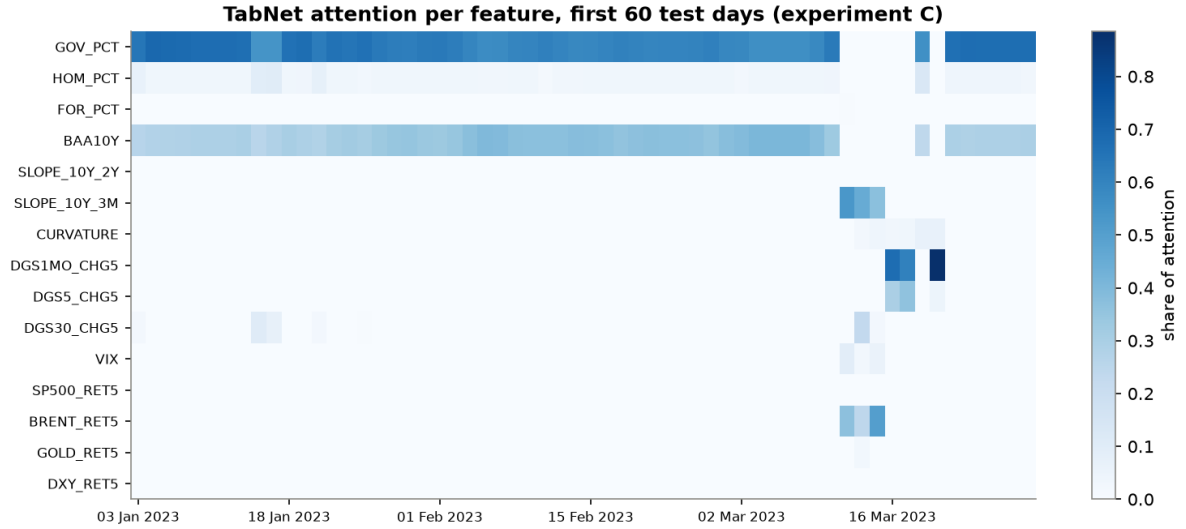


Figure 14: *TabNet attention masks for the first 60 test days (experiment C)*

TabNet's attention masks show which features it used on each day (Figure 14). For most of January to March 2023 it looked mainly at the slow-moving Fed debt share and the credit spread, but around mid-March 2023, when Silicon Valley Bank collapsed and yields fell sharply, its attention switched to the 10Y-3M slope and the recent changes of the 1M and 5Y yields. A normal MLP cannot show this.

Table 13: Top 5 features of each model in experiment C, using each framework's own method

Rank	Linear	XGBoost	Keras	fastai	TabNet	Flash
#1	SLOPE_10Y_3M	GOV_PCT	DGS30_CHG5	SLOPE_10Y_3M	SLOPE_10Y_3M	GOLD_RET5
#2	SLOPE_10Y_2Y	DGS5_CHG5	SLOPE_10Y_3M	SLOPE_10Y_2Y	GOV_PCT	CURVATURE
#3	DGS30_CHG5	DGS1MO_CHG5	SP500_RET5	GOLD_RET5	CURVATURE	SLOPE_10Y_2Y
#4	CURVATURE	SP500_RET5	DGS5_CHG5	GOV_PCT	DGS5_CHG5	BAA10Y
#5	GOV_PCT	SLOPE_10Y_3M	FOR_PCT	BAA10Y	BRENT_RET5	HOM_PCT

The 10Y-3M slope was among the top five features for five of the six models. It reflects how many interest-rate moves the market expects, which makes it the most useful single input. For the linear model the two slope features had large coefficients of opposite sign, so the model effectively used the 2Y-3M spread: when 2-year yields are above 3-month yields the market expects the Fed to raise rates, and the model predicts short-term yields to rise. The importances of the neural networks were very small (shuffling any single feature changed the test RMSE by less than 0.4 bp), consistent with networks that stayed close to the naive forecast.

4.7 Simplicity, transparency and efficacy

Table 14: Simplicity measures

Model	Lines of code	Hyperparameters set	Train time, one run (s)	Setup effort (1-3)
Naive	1	0	0.00	1
Linear	4	0	0.00	1
XGBoost	5	7	0.80	1
Keras	13	7	1.92	1
fastai	15	4	0.90	1
TabNet	6	10	9.24	1
Flash	15	4	3.01	3

Table 15: Transparency checklist (1 point each, 0.5 if partly)

Model	Global importance	Single prediction	Inspectable model	Same every run	Points
Linear	yes	yes	yes	yes	4
XGBoost	yes (gain)	yes (SHAP)	yes (trees)	yes	4
Keras	no	no	no	no	0
fastai	no	no	no	no	0
TabNet	yes (masks)	yes (masks)	partly	no	2.5
Flash	no	no	no	no	0

Table 16: Final scores of the five frameworks (1 = worst, 5 = best)

Framework	Simplicity	Transparency	Efficacy	Overall
XGBoost	4.0	5.0	5.0	4.67
TabNet	2.4	3.5	2.5	2.79
Keras	3.0	1.0	3.5	2.50

Framework	Simplicity	Transparency	Efficacy	Overall
fastai	3.4	1.0	2.0	2.13
Flash	2.3	1.0	2.0	1.75

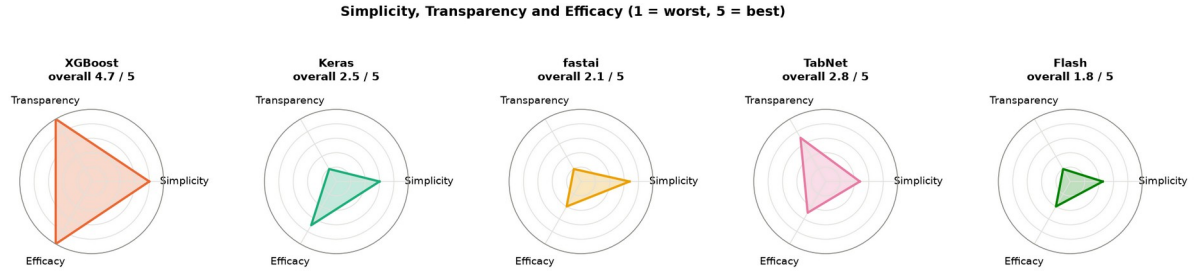


Figure 15: *Simplicity, transparency and efficacy of the five frameworks*

- **XGBoost (4.7/5)** needed the least code (5 lines), trained in under a second, is fully transparent (gain importance, exact SHAP values, the same result every run) and had the best results of the five frameworks.
- **TabNet (2.8/5)** is the second most transparent thanks to its attention masks, but it was the slowest (about 9 s per run), had the most hyperparameters (10) and the weakest results in experiment C.
- **Keras (2.5/5)** was easy to install and had a clear 13-line model with the second-best results, but it is a black box and depends on the random seed.
- **fastai (2.1/5)** normalises the inputs automatically and has a learning-rate finder, but it is a black box and its results were weaker.
- **Lightning Flash (1.8/5)** has short code, but because it is archived it needed a separate Python environment, old pinned versions and two fixes, and it has an in-place data bug (Section 4.8).

4.8 Practical problems and how they were solved

Table 17: Problems met during the project

Problem	Cause	Solution
Yahoo Finance has no 1M yield or debt data	Yahoo only lists four Treasury indices	Curve and debt data from FRED; Yahoo for market features and a cross-check
FRED download failed with an SSL certificate error	The python.org build of Python on macOS does not use the system certificates	Download with the requests library, which has its own certificates
PyTorch froze at 0% CPU after XGBoost had been used in the same process	XGBoost and PyTorch each ship their own OpenMP library on macOS	Every framework in its own notebook; the comparison notebook only reads CSV files
Lightning Flash could not be installed with the other libraries	Flash is archived and needs pytorch-lightning below 2.0	Separate Python 3.10 environment with pinned versions
Flash failed on import: No module named pkg_resources, then requests	setuptools 81+ removed pkg_resources; Flash does not declare requests	Pinned setuptools below 81 and added requests
Flash forecasts were wrong by up to hundreds of percentage points	Flash normalises the DataFrames passed to it in place, so reused data was normalised again and again	Pass a copy of every DataFrame to Flash

The Flash bug was found because the first results were impossible (errors far larger than any yield), and only some random seeds were affected. Comparing seeds showed that the normalisation statistics changed between runs on the same data, and reading Flash's source code showed the in-place operation. Because Flash is archived, this bug will not be fixed by its developers. It is a good example of the risk of building on an abandoned library.

5. Conclusion and Summary

5.1 Summary of findings

1. **The target matters more than the framework.** Predicting the yield level, as in the original case study, made every model fail on 2023-2026 because yields reached levels never seen in training. Trees failed worst because they cannot extrapolate. Predicting the weekly change and removing the raw level inputs brought every model to within about 1.7 bp of the naive forecast.
2. **No model beat the naive "no change" forecast** in a statistically significant way. One-week-ahead Treasury yield moves are close to unpredictable from public data, which is consistent with efficient markets and with earlier research.
3. **Simpler is better here.** Linear regression and XGBoost came closest to the naive forecast; the four neural networks were slightly worse and depended on the random seed.
4. **R^2 on yield levels is misleading** for this problem, because even the naive forecast scores 0.84-0.96. The skill against the naive forecast and the R^2 on the change are better measures.
5. **Framework ranking:** XGBoost first (simple, transparent and best results), then TabNet, Keras, fastai and Lightning Flash. For a real system we would use XGBoost or a linear model on the change target and always check it against the naive benchmark.

5.2 Limitations

- Only one forecast horizon (one week) and one train / validation / test split were studied. The models were trained once on 2010-2021 and never updated with newer data.
- The publication delay of the debt data was approximated with a fixed lag of two or three quarters.
- The tuning budget was small (4 to 12 configurations per framework) and everything ran on a CPU.
- No inflation, employment or Fed funds futures data was used.
- Parts of the simplicity and transparency scores (setup effort, the checklist) involve judgement.

5.3 Future work

- Longer horizons (one to three months), where economic data may have more predictive power.
- Forecasting the Nelson-Siegel level, slope and curvature factors instead of single tenors (Diebold & Li, 2006).
- Retraining the models regularly (walk-forward validation) so that they adapt to new interest-rate regimes.
- Adding market expectations of the policy rate (Fed funds futures) and inflation data.

References

[1] Arik, S. Ö., & Pfister, T. (2021). TabNet: Attentive interpretable tabular learning. Proceedings of the AAAI Conference on Artificial Intelligence, 35(8), 6679-6687.

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.

[3] Diebold, F. X., & Li, C. (2006). Forecasting the term structure of government bond yields. Journal of Econometrics, 130(2), 337-364.

[4] Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. Journal of Business & Economic Statistics, 13(3), 253-263.

[5] Estrella, A., & Mishkin, F. S. (1998). Predicting U.S. recessions: Financial variables as leading indicators. Review of Economics and Statistics, 80(1), 45-61.

[6] Federal Reserve Bank of St. Louis. FRED, Federal Reserve Economic Data. https://fred.stlouisfed.org

[7] Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? Advances in Neural Information Processing Systems 35 (Datasets and Benchmarks Track).

[8] Howard, J., & Gugger, S. (2020). Fastai: A layered API for deep learning. Information, 11(2), 108.

[9] Joseph, M. (2021). PyTorch Tabular: A framework for deep learning with tabular data. arXiv:2104.13638.

[10] Litterman, R., & Scheinkman, J. (1991). Common factors affecting bond returns. The Journal of Fixed Income, 1(1), 54-61.

[11] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems 30.

[12] Newey, W. K., & West, K. D. (1987). A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. Econometrica, 55(3), 703-708.

[13] Smith, L. N. (2018). A disciplined approach to neural network hyper-parameters: Part 1. arXiv:1803.09820.

[14] Tatsat, H., Puri, S., & Lookabaugh, B. (2020). Machine Learning and Data Science Blueprints for Finance. O'Reilly Media.

[15] Yahoo Finance, accessed with the yfinance Python library. https://github.com/ranaroussi/yfinance

Appendix A: Project structure and how to run

Table 18: Contents of the GitHub repository

Path	Contents
src/config.py	Dates, tickers, split dates, experiments, random seeds
src/data.py	Downloading from FRED and Yahoo Finance, building the daily dataset
src/features.py	Feature engineering, targets, time-based split, scaling
src/evaluate.py	Metrics, saving predictions, permutation importance
src/plots.py	Common plot style and colours
notebooks/01_data_collection.ipynb	Data download, preparation and checks
notebooks/02_eda.ipynb	Exploratory data analysis
notebooks/03_baselines.ipynb	Naive forecast and linear regression
notebooks/04_xgboost.ipynb … 08_lightning_flash.ipynb	One notebook per framework
notebooks/09_comparison.ipynb	Comparison of all models and the final scores
data/raw, data/processed	Downloaded series (frozen at 30 Sep 2026) and the final dataset
results/	Predictions, metrics, importances and all figures
requirements.txt, requirements-flash.txt	The two Python environments

To reproduce: create the main environment with Python 3.12 and install requirements.txt; create the Flash environment with Python 3.10 and install requirements-flash.txt; register both as Jupyter kernels; then run notebooks 01 to 09 in order (notebook 08 uses the flash-env kernel). Every notebook only needs the files written by the notebooks before it.

Appendix B: Chosen hyperparameters

Hyperparameters were chosen on the 2022 validation set with seed 42.

Table 19: XGBoost (one model per target; subsample = colsample_bytree = 0.8, early stopping after 100 rounds)

Experiment	Target	max_depth	learning_rate	min_child_weight	Trees kept
A	1M	3	0.05	1	115
A	5Y	2	0.05	20	144
A	30Y	3	0.05	20	115
B	1M	5	0.01	20	6
B	5Y	5	0.05	1	11
B	30Y	5	0.05	1	1
C	1M	2	0.01	20	1
C	5Y	3	0.05	20	242
C	30Y	5	0.05	1	174

Table 20: Neural networks

Framework	Experiment A	Experiment B	Experiment C
Keras (Adam, lr 0.001)	layers (32), no dropout	layers (32), no dropout	layers (64, 32), no dropout
fastai	[64, 32], lr 0.01, 30 epochs	[64, 32], lr 0.001, 10 epochs	[64, 32], lr 0.001, 10 epochs
TabNet	n_d = n_a = 8, 5 steps	n_d = n_a = 8, 3 steps	n_d = n_a = 16, 3 steps
Lightning Flash	lr 0.001, 10 epochs	lr 0.001, 30 epochs	lr 0.01, 10 epochs